



EC2



Load Balancer



VPC



S3



PuTTY



docker

PROJECT DOCUMENTATION:

LAUNCH YOUR Linux MACHINE ON AWS WITH PUTTY



Step-by-Step
Guide



Secure
Connection



AWS
Infrastructure



Real-World
Use Case

Learn | Build | Deploy

▪ PROJECT OVERVIEW:

The goal of this project is to provision a virtual Ubuntu Linux server using Amazon EC2 on **AWS** and establish a secure, remote command-line connection to it from a local machine via **PuTTY**. The workflow begins by launching the Linux instance on AWS and downloading its cryptographic security key pair (.pem). To allow remote access, a firewall rule is configured in the AWS Security Group by opening **Port 22** for SSH traffic. Next, the downloaded key is converted into a PuTTY-compliant format (.ppk) using the **PuTTYgen** utility. Finally, by inputting the AWS instance's Public IP address and attaching the converted key file into **PuTTY**, a secure, passwordless authentication handshake is completed, granting direct control over the remote cloud terminal 🚀.

▪ PREREQUISITES 🔑:

Before launching your Linux machine on AWS and connecting via PuTTY, ensure you have the following prerequisites ready:

- **AWS Account** : An active Amazon Web Services account with permissions to launch EC2 instances.
- **PuTTY Installed** : The PuTTY desktop application installed on your local machine (includes both PuTTY and PuTTYgen).
- **Network Access** 🌐 : A stable internet connection to access the AWS Management Console and establish the SSH connection.
- **Key Pair** 🔑 : Access to download the .pem private key file generated by AWS during instance setup.
- **Security Group Rules** : Awareness of your local public IP address to restrict **Port 22** access to your machine only (recommended for security).

 **Command Line Familiarity** : Comfortable using basic Linux commands (e.g., ls, cd, pwd).

Once you've got these, you're all set to launch your AWS Linux instance and connect using PuTTY!

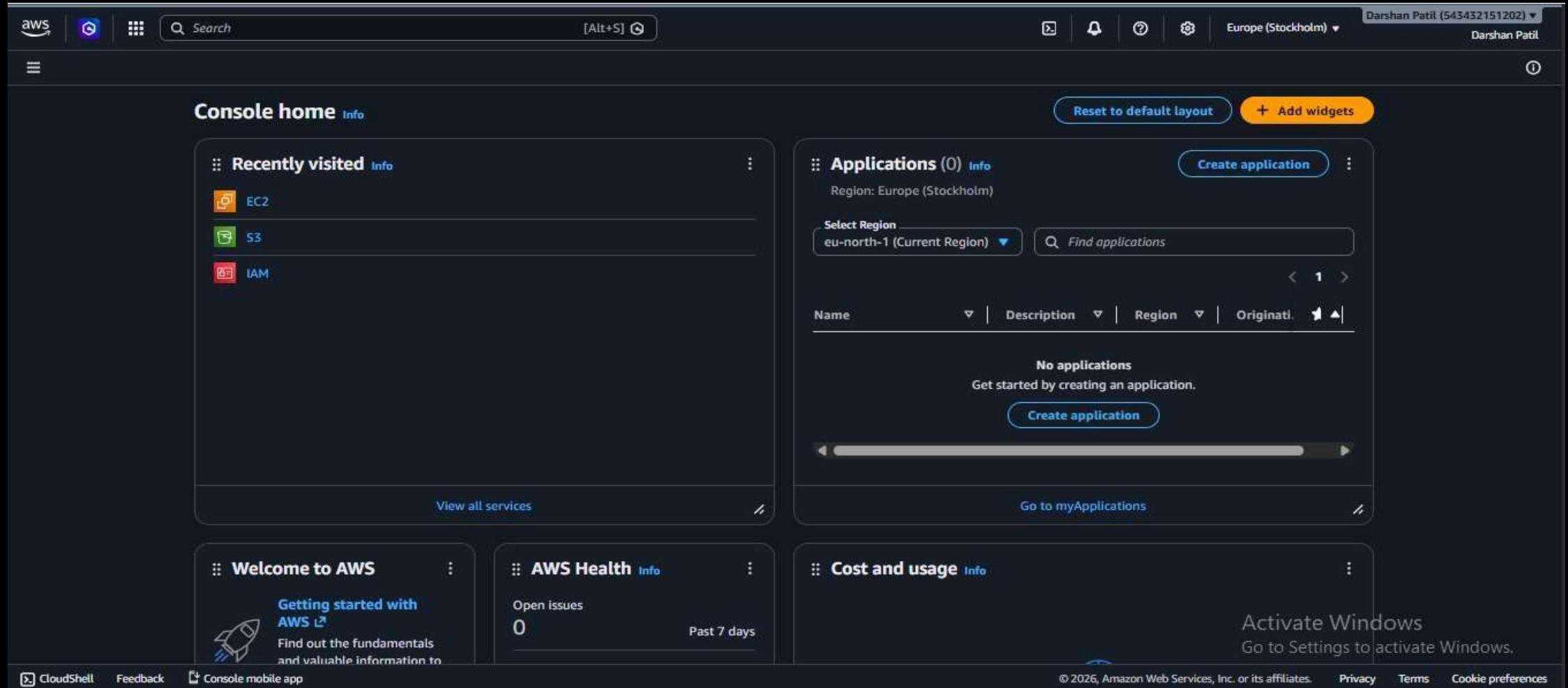
▪ WHAT IS PUTTY?

PuTTY is a free and open-source SSH client that allows you to securely connect to remote servers, typically Linux-based, from a Windows machine. It enables you to access and manage your server via the command line.  

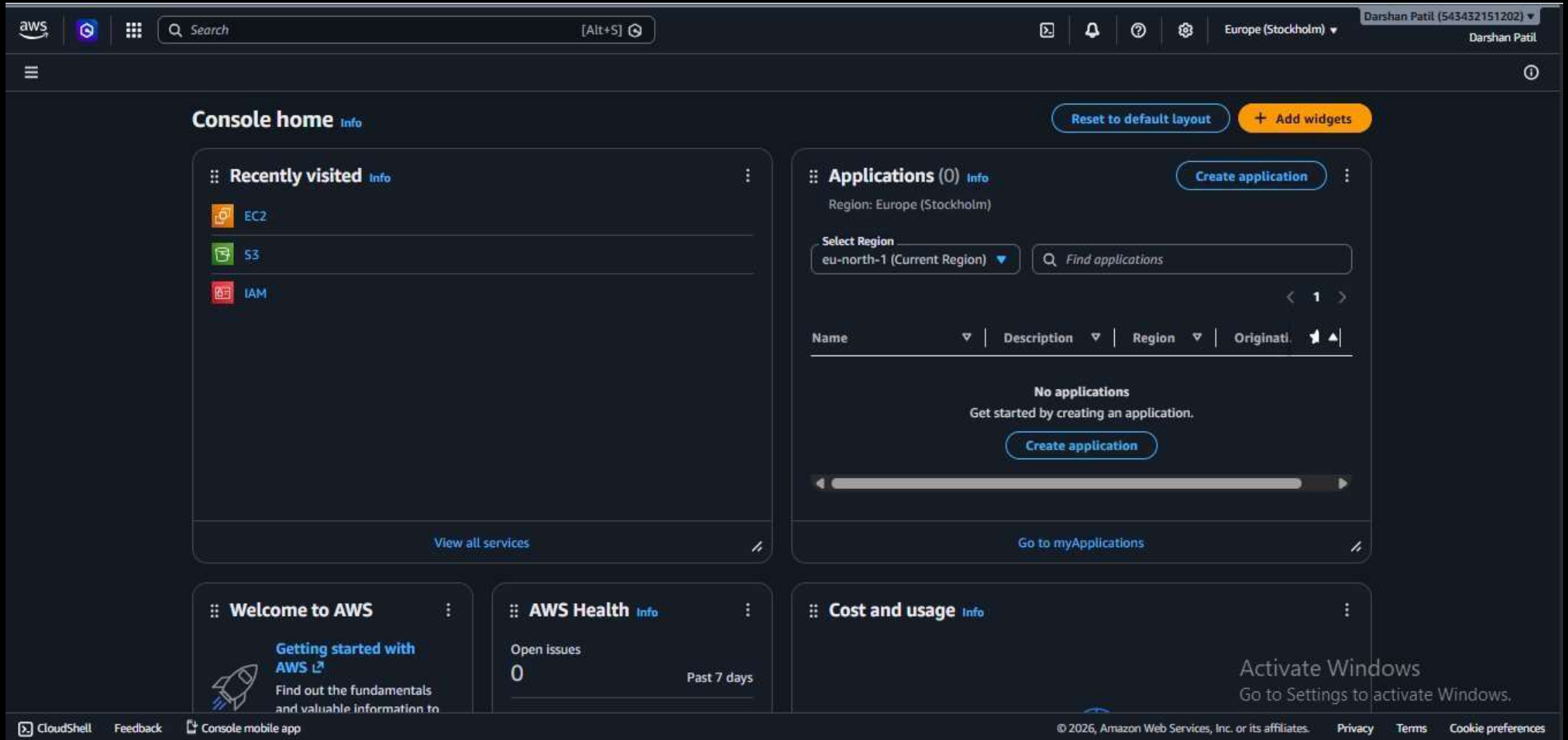
Ready to get your Linux server running on AWS and connect to it using PuTTY? Let's go through the steps together in a fun and easy way! 

I. LAUNCH YOUR EC2 INSTANCE ON AWS 🚀

1. Log in to your **AWS Management Console**. It's time to get started.



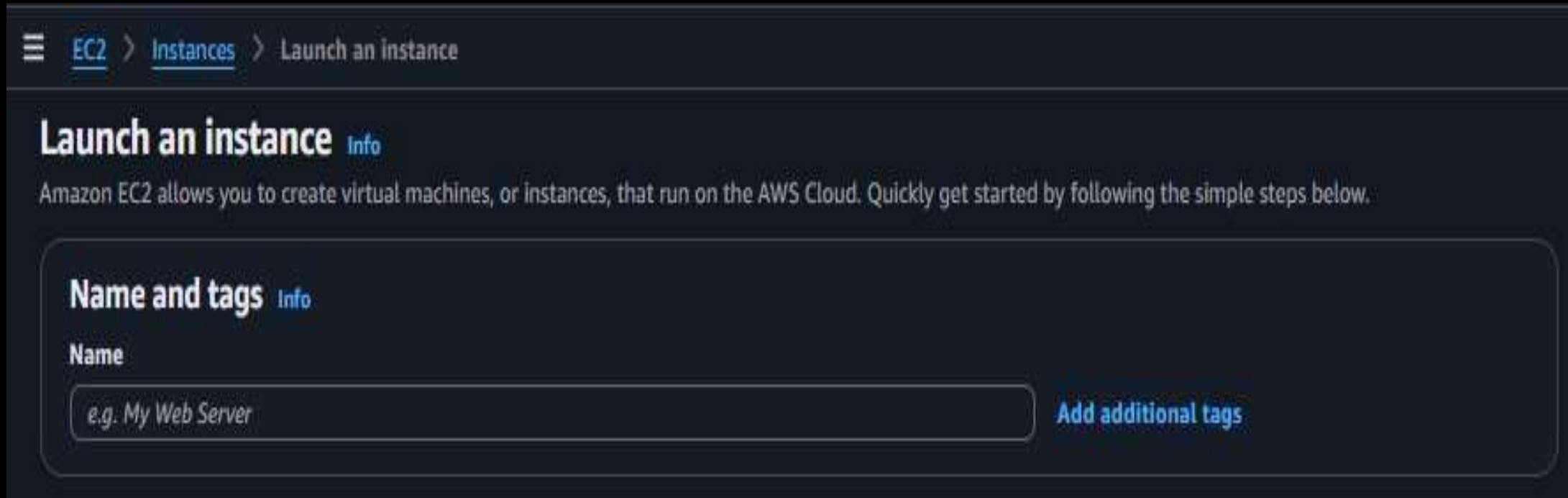
2. Head over to **EC2 Dashboard** under **Services** , the heart of your cloud adventure



3. Click the **Launch Instance** button and get ready to takeoff ✈️



4. Give a name to your instance , this helps you easily identify it later. 💡
(Eg – creating-linux-using-putty)



The screenshot shows the 'Launch an instance' page in the AWS Management Console. The breadcrumb navigation at the top reads 'EC2 > Instances > Launch an instance'. The main heading is 'Launch an instance' with an 'Info' link. Below this is a descriptive paragraph: 'Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.' The first step is 'Name and tags', also with an 'Info' link. Under this step, there is a 'Name' label and a text input field containing the placeholder text 'e.g. My Web Server'. To the right of the input field is a button labeled 'Add additional tags'.

5. Pick your Amazon Machine Image (AMI) (e.g... Ubuntu or Amazon Linux)

▼ **Application and OS Images (Amazon Machine Image)** [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Recents

Quick Start

Amazon Linux



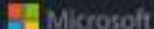
macOS



Ubuntu



Windows



Red Hat



SUSE Linux



Debian





[Browse more AMIs](#)

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

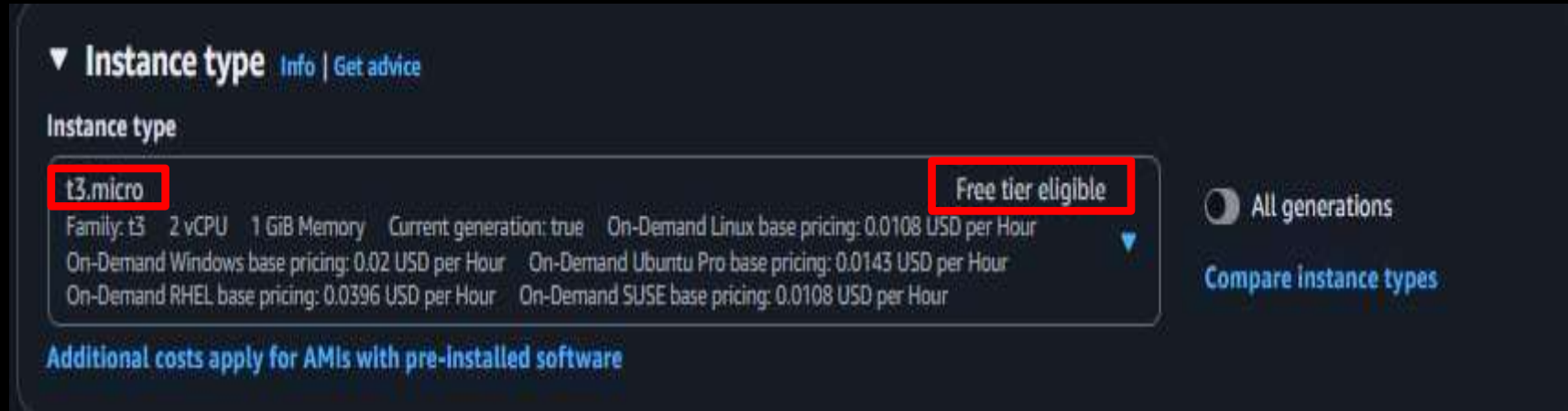
Amazon Linux 2023 kernel-6.18 AMI

ami-0402e980e69d5978b (64-bit (x86), uefi-preferred) / ami-033fd902867ca7065 (64-bit (Arm), uefi)

Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible ▼

6. **Choose an Instance Type** : Select **t2.micro** , which is perfect for beginner and falls within free tier offering 🎁.



▼ **Instance type** [Info](#) | [Get advice](#)

Instance type

t2.micro **Free tier eligible**

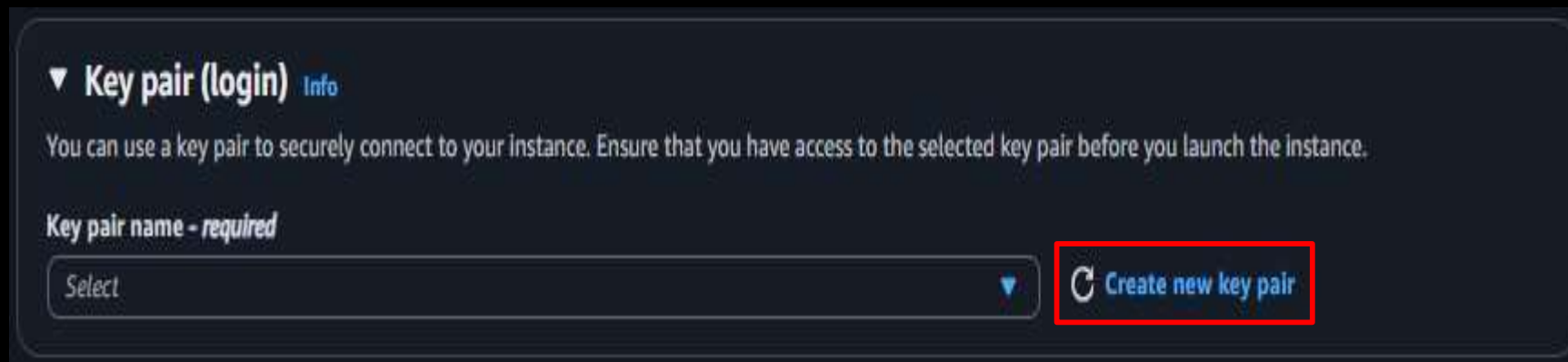
Family: t2 2 vCPU 1 GiB Memory Current generation: true On-Demand Linux base pricing: 0.0108 USD per Hour
On-Demand Windows base pricing: 0.02 USD per Hour On-Demand Ubuntu Pro base pricing: 0.0143 USD per Hour
On-Demand RHEL base pricing: 0.0396 USD per Hour On-Demand SUSE base pricing: 0.0108 USD per Hour

☐ All generations

[Compare instance types](#)

[Additional costs apply for AMIs with pre-installed software](#)

7. In the **Key Pair** section, click on **Create a new pair** to generate a new key pair for secure access 🔑



▼ **Key pair (login)** [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*

Select ▼

[Create new key pair](#)

8. Give your key pair a name (e.g.. **my-key-pair**) 📄

Create key pair

Key pair name
Key pairs allow you to connect to your instance securely.

my-key-pair

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ **RSA**
RSA encrypted private and public key pair

☐ **ED25519**
ED25519 encrypted private and public key pair

Private key file format

☒ **.pem**
For use with OpenSSH

☐ **.ppk**
For use with PuTTY

⚠ When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. [Learn more](#) ↗

Cancel Create key pair

9. Configure your **Security Group** , just make sure **SSH** (port 22) is allowed 🔒.

▼ **Network settings** [Info](#)

Network [Info](#)

vpc-0ca3dd3c304fb2fcd

Subnet [Info](#)

No preference (Default subnet in any availability zone)

Auto-assign public IP [Info](#)

Enable

Firewall (security groups) [Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group

☐ Select existing security group

We'll create a new security group called 'launch-wizard-5' with the following rules:

☒ Allow SSH traffic from
Helps you connect to your instance

Anywhere
0.0.0.0/0

☐ Allow HTTPS traffic from the internet
To set up an endpoint, for example when creating a web server

☐ Allow HTTP traffic from the internet
To set up an endpoint, for example when creating a web server

⚠️ Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

×

10. See the instance **Summary** and click on **Launch Instance** 🖱️

▼ **Summary**

Number of instances | Info

1

Software Image (AMI)

-

Virtual server type (instance type)

-

Firewall (security group)

-

Storage (volumes)

-

Cancel

Launch instance

🔗 Preview code

What is PuTTYgen ? 🔑

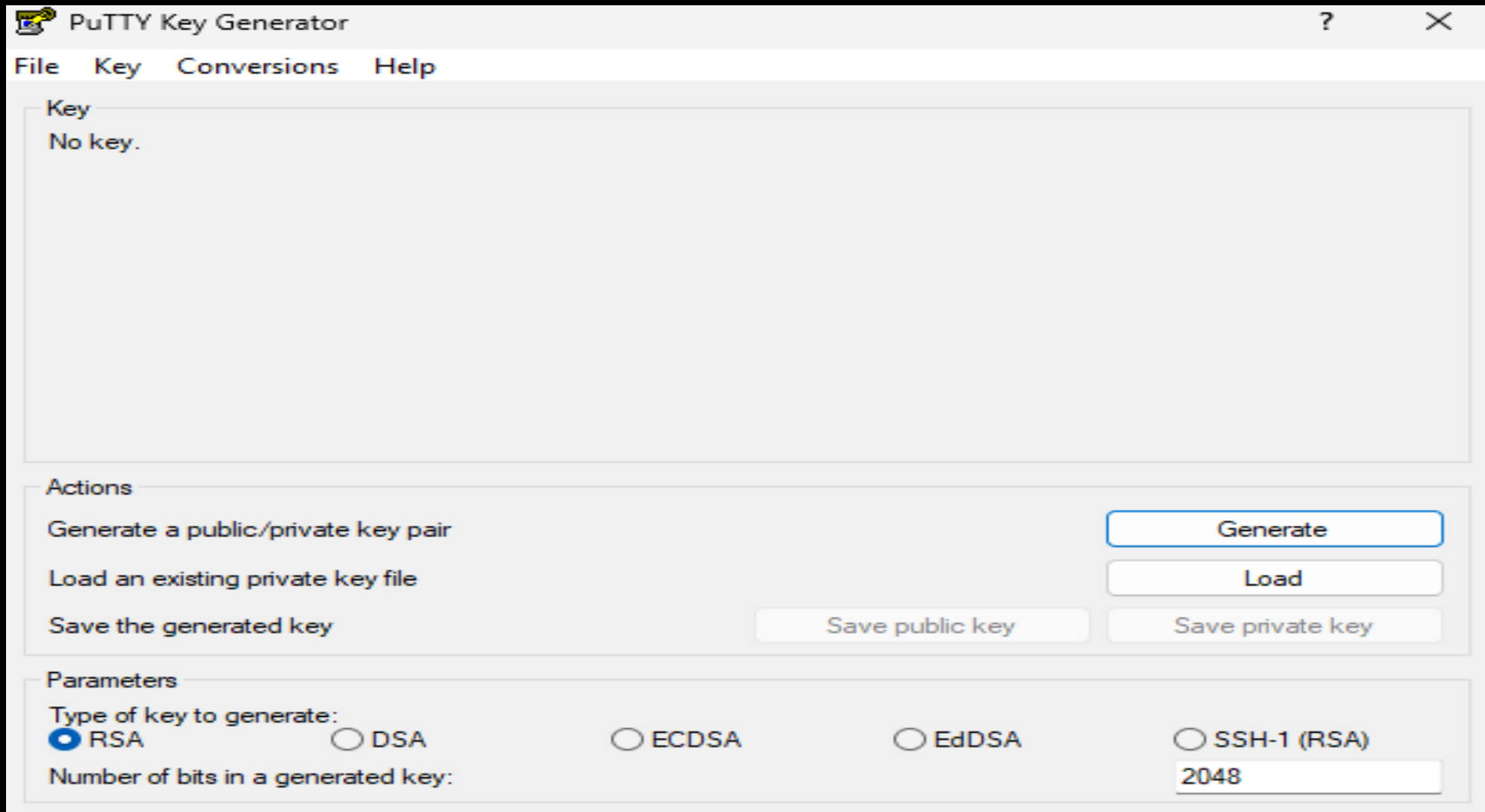
PuTTYgen is a tool used to generate and convert SSH key pairs. It allows you to convert AWS EC2 key pairs (.pem files) into PuTTY-compatible format (.ppk), which is required for securely connecting to your server via PuTTY. 🔒

Why We Use PuTTYgen? 🔑

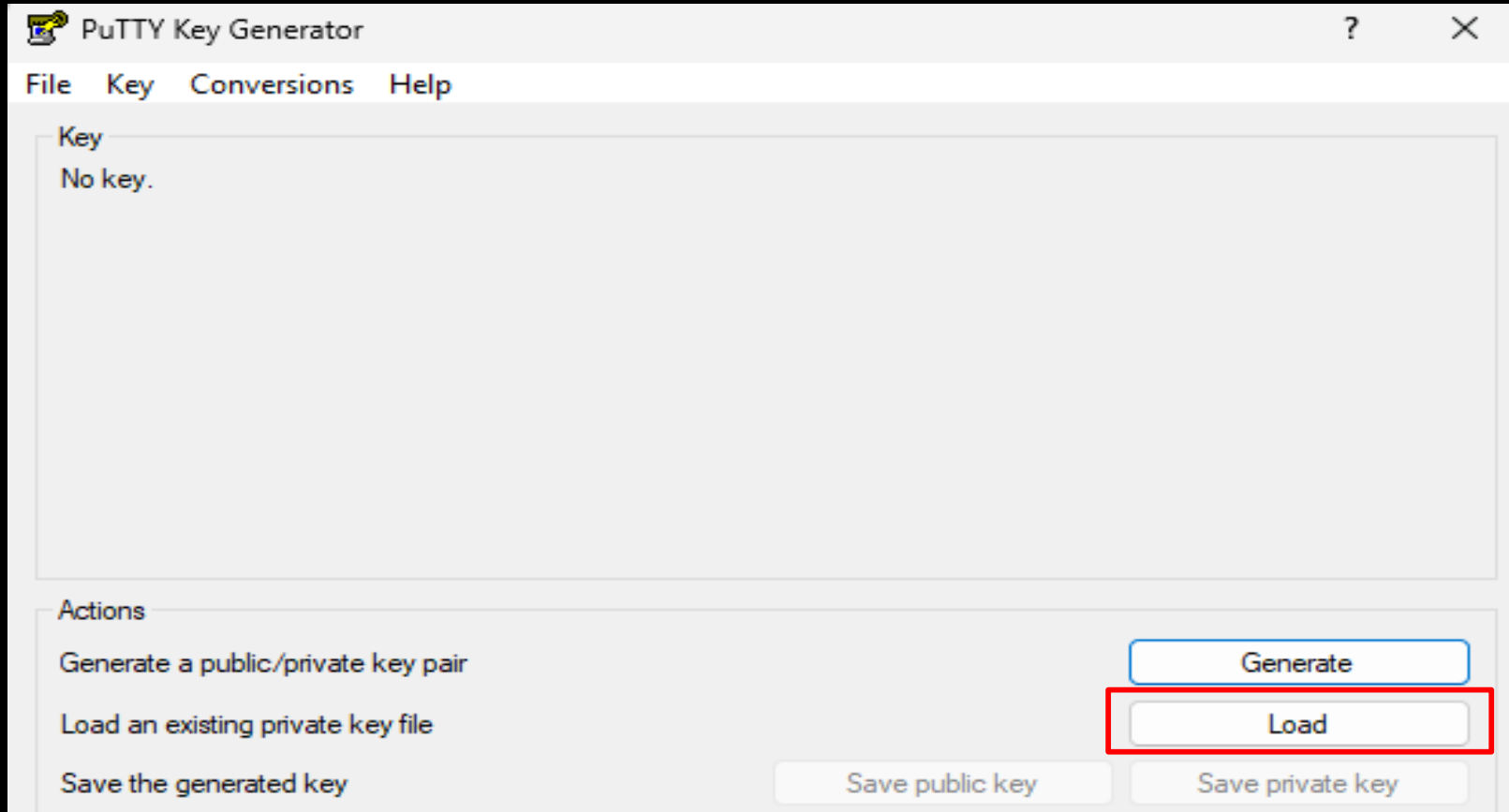
We use PuTTYgen to convert AWS EC2 key pairs (.pem files) into a PuTTY-compatible format (.ppk), allowing us to securely connect to a remote Linux server using PuTTY. Without this conversion, PuTTY cannot authenticate the connection. 📞

II. CONVERT YOUR .PEM FILE TO .PPK USING PUTTYGEN

1. Open PuTTYgen, your magical key converter! ✨



2. Click **Load** and select your freshly downloaded .pem.file. 🔒



 PuTTY Key Generator ? X

File Key Conversions Help

Key

Public key for pasting into OpenSSH authorized_keys file:

```
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCE73rSlo1OQu7qswnlgeBCBsbbbMRVqnYYbYW+re
+7eJyINQPNjCrRaV01z6+zld41ZnVTSvmzsGSzL91b621fi0jknMBzmGUJtxo6cDHrxSgfH/86svhzD8zGyGVsRRPr
7xdPQWWISpu5wSbH08t1865kxshDMOWjGxaC1amQO4CtxnaLRx72CgtsvZA3pt/1vvUbbOkeyyvYcJ8Fov
+an8L0aQqTD083cL8SA1mQZMaUnCeh
+23Fo0GuzcV4Qa6kUH0rWhCkdTLx2Plu721yUysjgWX/1gPtakOD7pa1HmkRaU6nXao9f4wyQlm73g6K/KxBub
```

Key fingerprint: `ssh-rsa 2048 SHA256:r5Bts2GPU6C17xpx7t2O+KVV852cZVvJXE9oh2SD30U`

Key comment: `imported-openssh-key`

Key passphrase:

Confirm passphrase:

Actions

Generate a public/private key pair Generate

Load an existing private key file Load

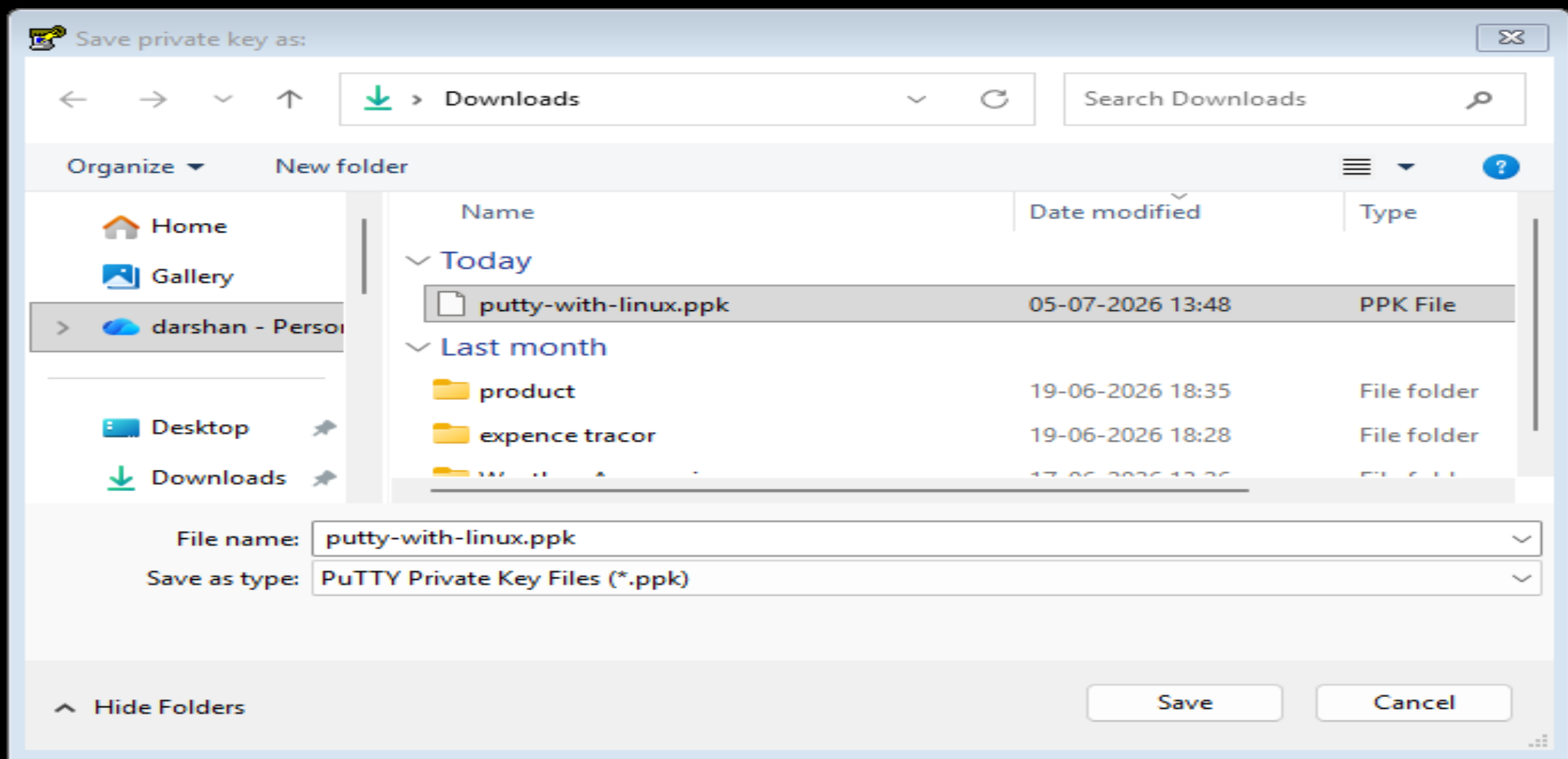
Save the generated key Save public key Save private key

Parameters

Type of key to generate:

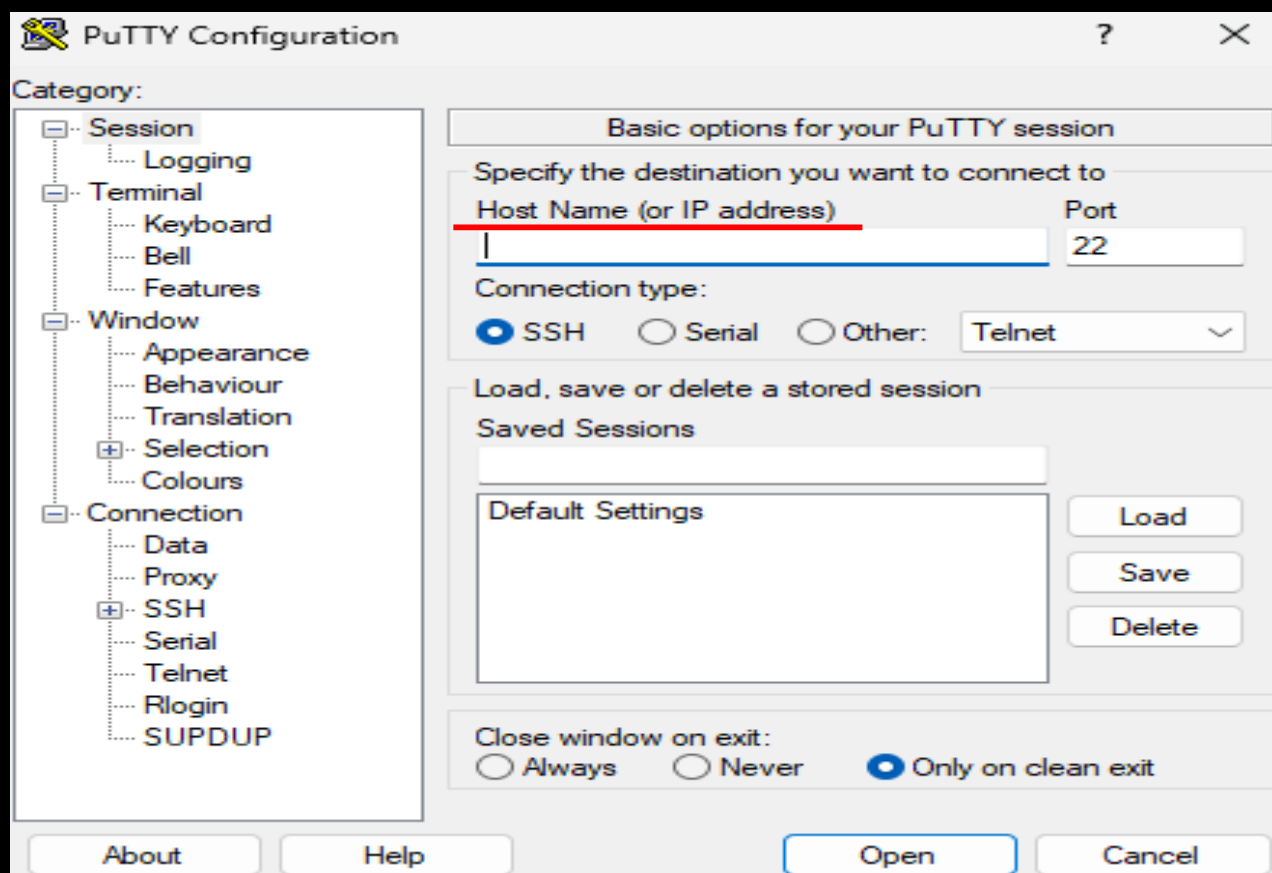
☒ RSA ☐ DSA ☐ ECDSA ☐ EdDSA ☐ SSH-1 (RSA)

Number of bits in a generated key: `2048`



III. CONNECT TO YOUR EC2 INSTANCE WITH PUTTY 📞

1. Open **PuTTY** and get ready to teleport into your EC2 instance. 🌐
2. In the **Host Name** field, enter the **Public IP Address** or **Public DNS** of your EC2 instance. 🚀



S3 EC2

EC2 > Instances > i-07c08217508a7e4af

EC2

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Instance summary for i-07c08217508a7e4af (linux-with-putty) Info

Updated less than a minute ago

Instance ID

i-07c08217508a7e4af

IPv6 address

-

Hostname type

IP name: ip-172-31-22-17.eu-north-1.compute.internal

Answer private resource DNS name

IPv4 (A)

Auto-assigned IP address

13.61.190.156 [Public IP]

IAM role

-

IMDSv2

Required

Operator

Public IPv4 address

13.61.190.156 | open address

Instance state

Running

Private IP DNS name (IPv4 only)

ip-172-31-22-17.eu-north-1.compute.internal

Instance type

t3.micro

VPC ID

vpc-0ca3dd3c304fb2fcd

Subnet ID

subnet-03637d372f6e57462

Instance ARN

arn:aws:ec2:eu-north-1:543432151202:instance/i-07c08217508a7e4af

Connect

Instance state

Actions

Private IPv4 addresses

172.31.22.17

Public DNS

ec2-13-61-190-156.eu-north-1.compute.amazonaws.com | open address

Elastic IP addresses

-

AWS Compute Optimizer finding

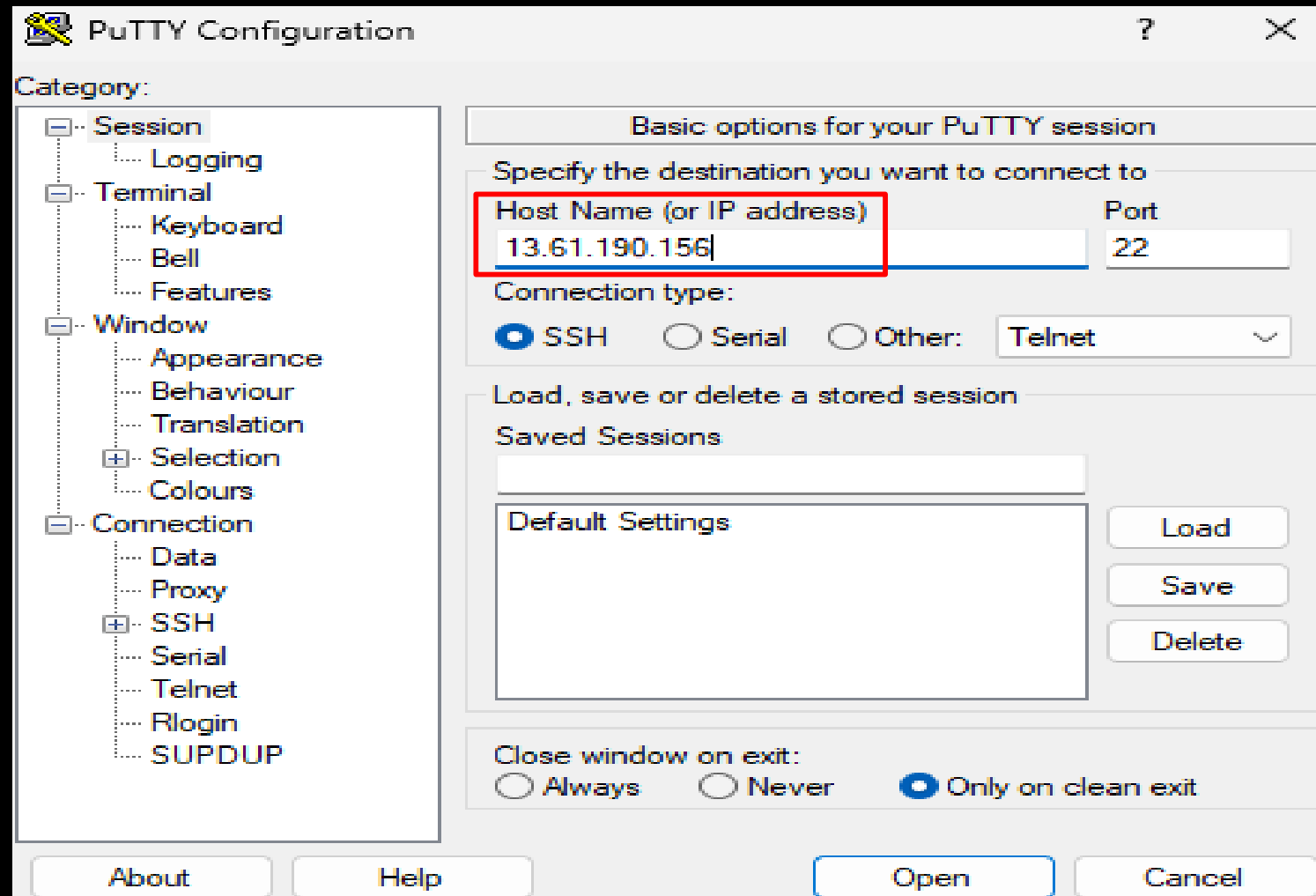
Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name

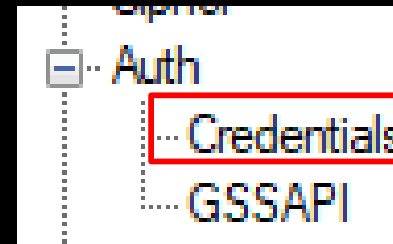
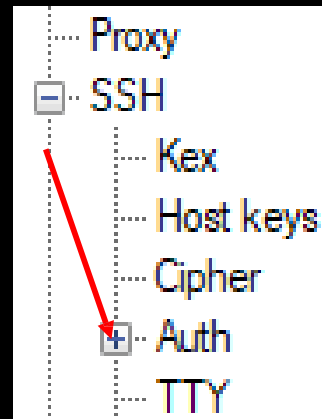
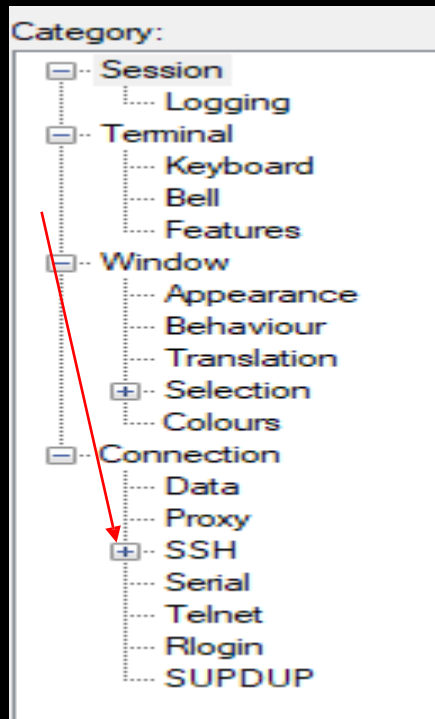
-

Managed

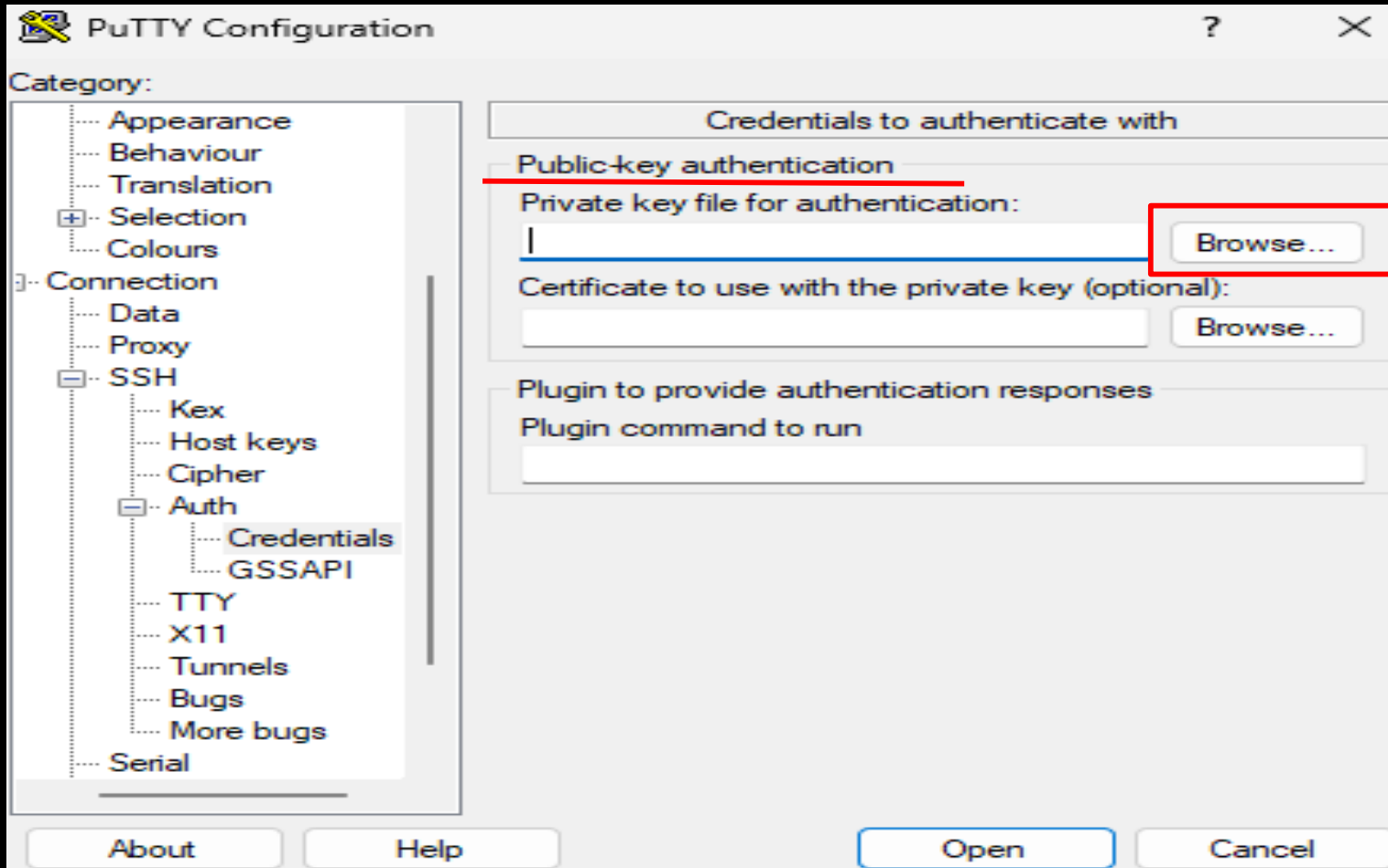
false

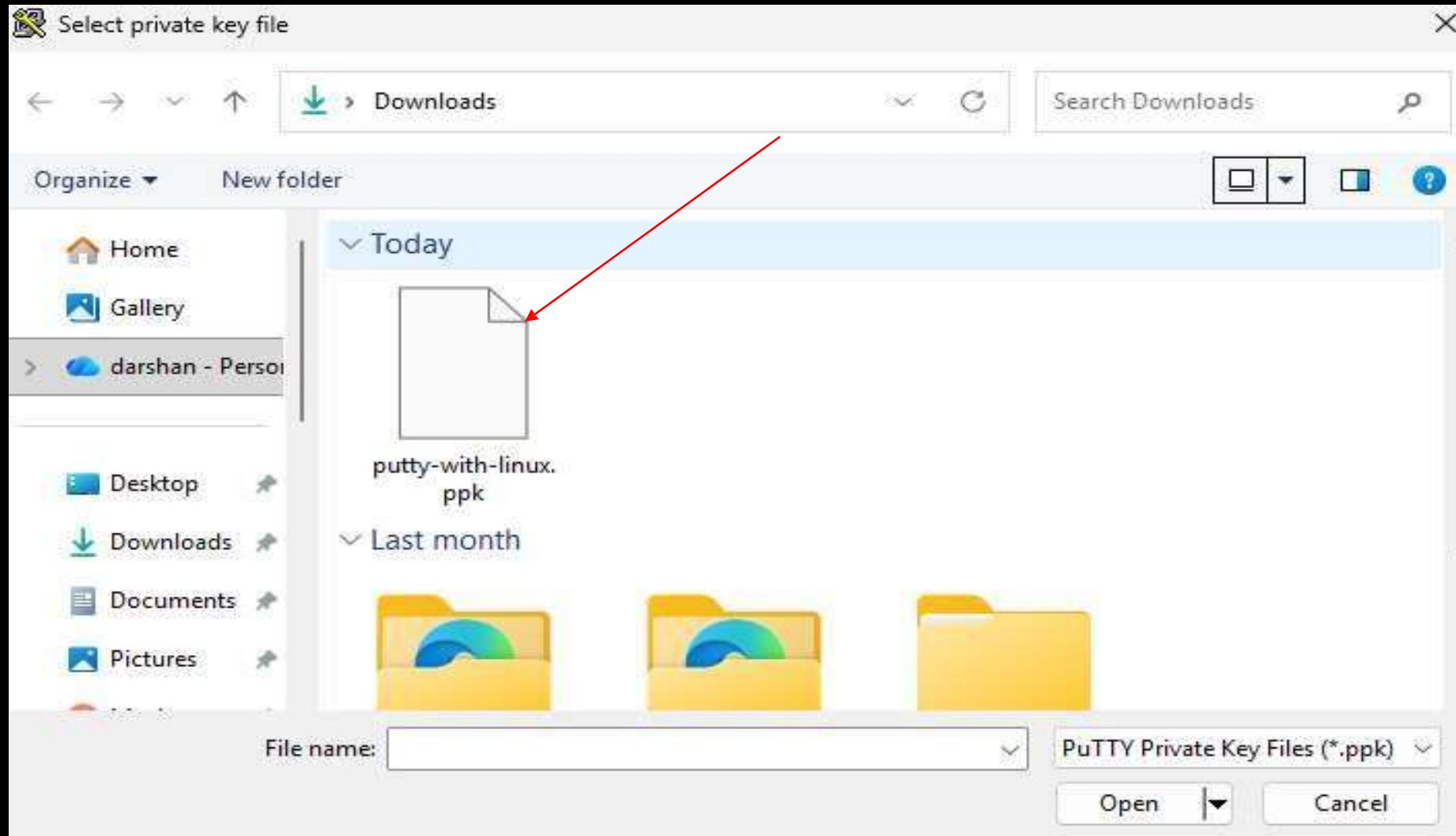


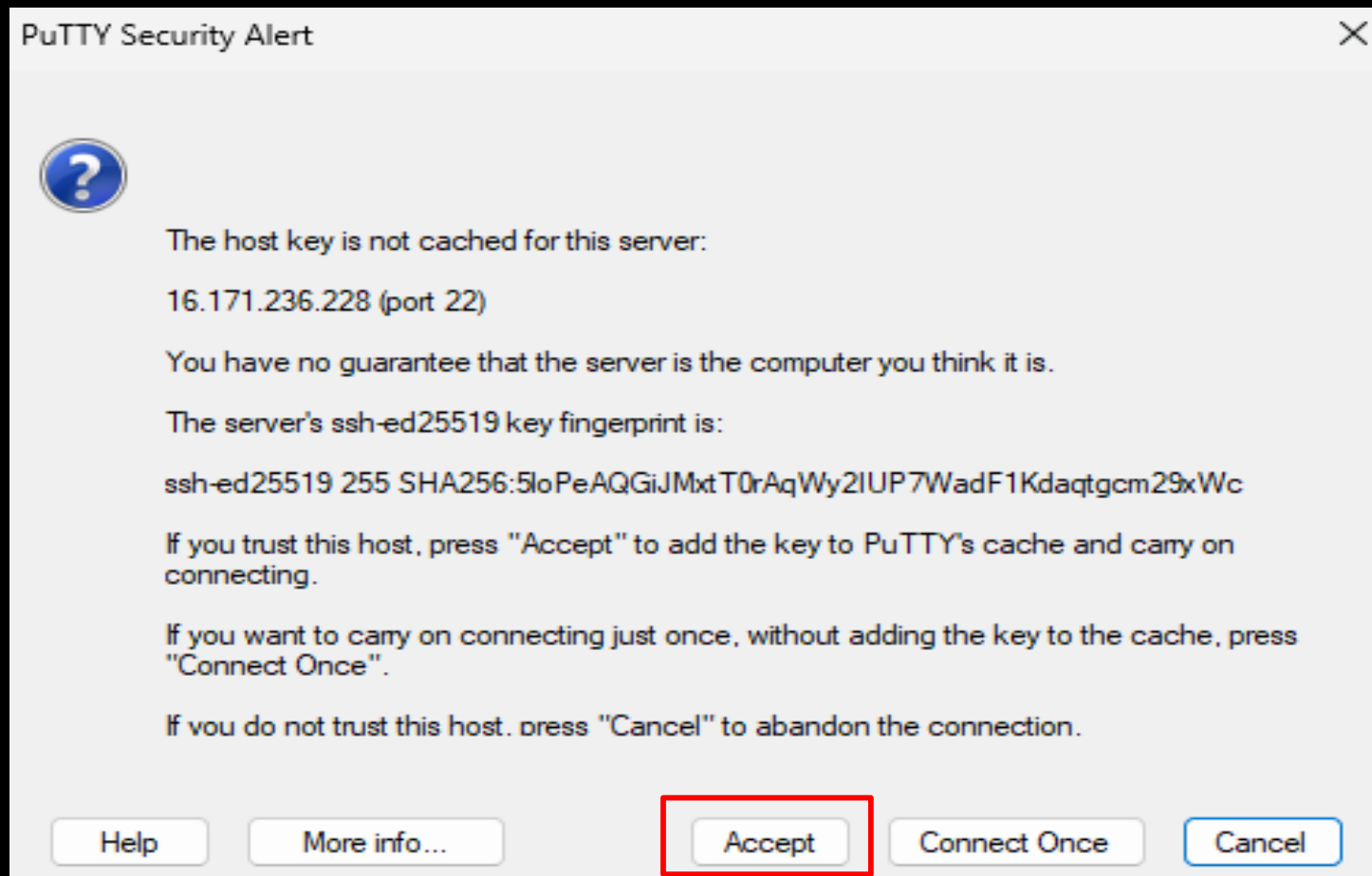
3. Go to **SSH > Auth** on the left menu.📁



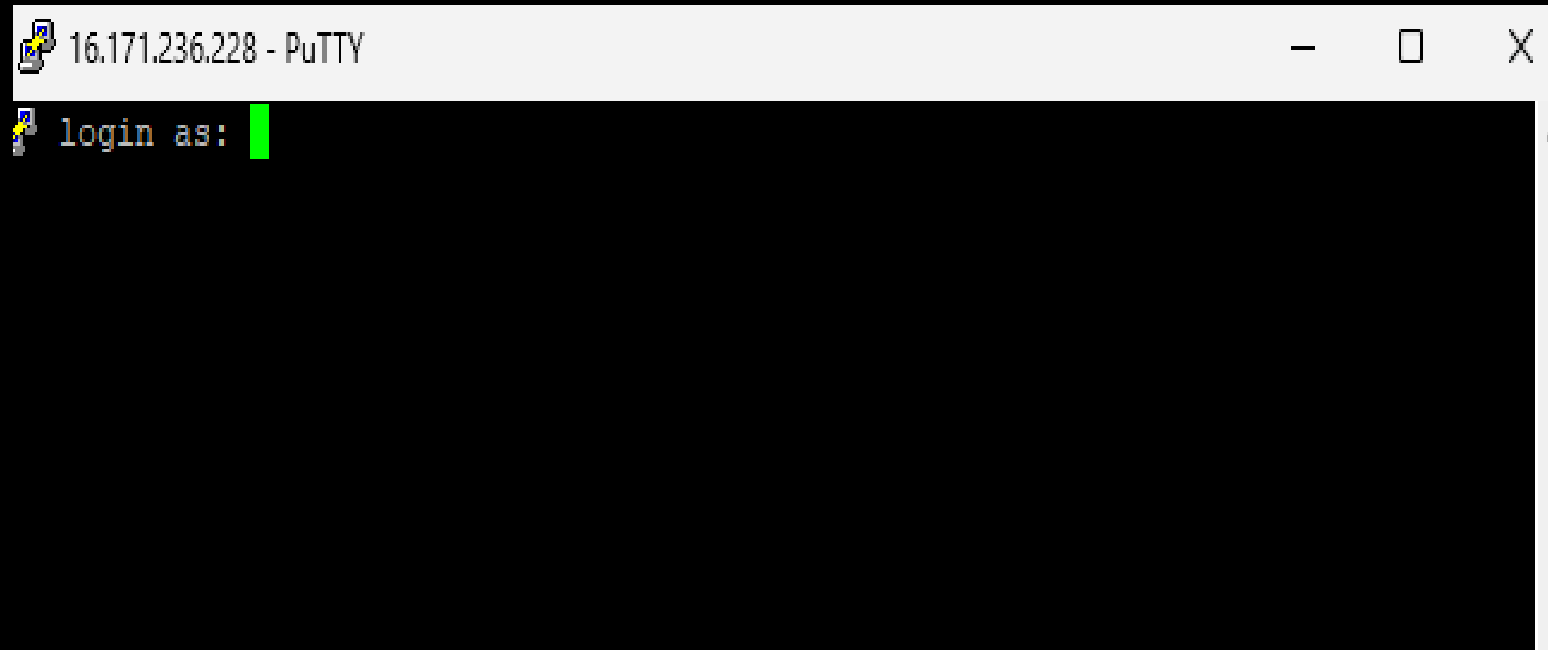
4 .Under Private key file for authentication, browse and select your .ppk file. 🔑







5. Hit **Open** and voilà, you're almost there!



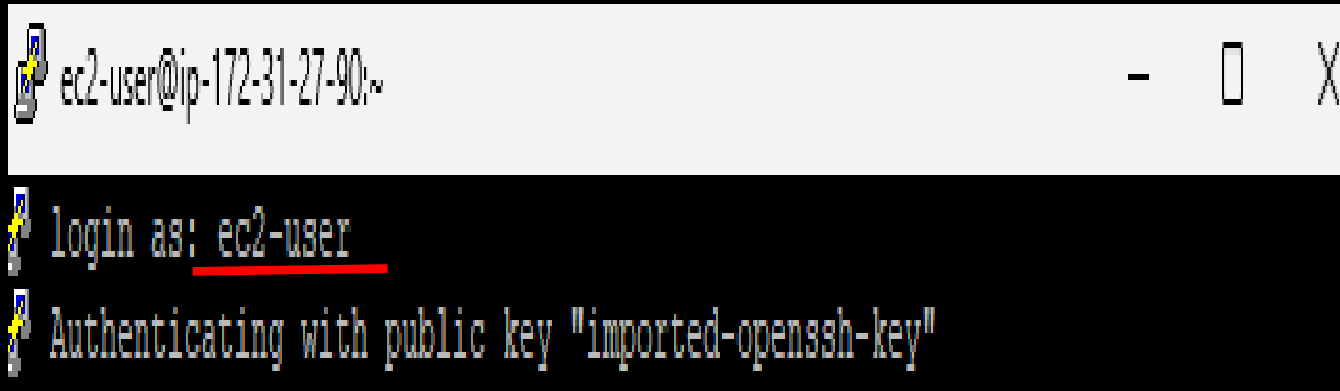
IV. LOGIN LIKE A PRO

1. Depending on your AMI, use the **default username** to log in: -

Amazon Linux: ec2-user

Ubuntu: ubuntu

CentOS: centos

A screenshot of a terminal window with a title bar containing a minimize icon, a maximize icon, and a close icon. The terminal text shows the user 'ec2-user' at the IP address 'ip-172-31-27-90'. The prompt is '~'. The user enters 'login as: ec2-user', which is underlined in red. The next line shows 'Authenticating with public key "imported-openssh-key"'.

```
ec2-user@ip-172-31-27-90:~  
login as: ec2-user  
Authenticating with public key "imported-openssh-key"
```




```
 ec2-user@ip-172-31-27-90:~  
login as: ec2-user  
Authenticating with public key "imported-openssh-key"  
  
#_      _____  
~\    #####          Amazon Linux 2023  
~~ \   #####\  
~~  \#####|  
~~   \|###|  
~~    \|#/       https://aws.amazon.com/linux/amazon-linux-2023  
~~     V~' '->  
~~~~  
~~ . _ . /  
~~/_/_/_/  
~~/_m/'
```

[ec2-user@ip-172-31-27-90 ~]\$ █